

AGENTS.md - HelixCode Authoritative Agent Guide

HelixCode Agent Guidelines

Version: 2.0.0 (Updated with CONST-035 Anti-Bluff Mandate) **Date:** 2026-04-30 **Scope:** All AI agents, human contributors, and automated processes working on HelixCode **Authority:** Derived from HelixAgent AGENTS.md with HelixCode-specific enhancements

Project Overview

HelixCode is an enterprise-grade distributed AI development platform built in Go that enables intelligent task division, work preservation, and cross-platform development workflows.

Current Status: Foundation is solid (auth system verified real), but CRITICAL bluff areas exist in core user-facing features. All agents MUST prioritize zero-bluff implementation.

Key Features (Target State):

- **Distributed Computing:** SSH-based worker pools with auto-installation and health monitoring
 - **Multi-Provider LLM Integration:** Support for 15+ providers
 - **Development Workflows:** Automated planning, building, testing, refactoring
 - **Task Management:** Intelligent task division with checkpointing
 - **MCP Protocol:** Full Model Context Protocol implementation
 - **Multi-Client Architecture:** REST API, CLI, Terminal UI, WebSocket, Desktop, Mobile
 - **Memory Systems:** Integration with external memory providers
 - **Advanced Editor:** Multi-format code editing
 - **Tools Ecosystem:** 40+ comprehensive tools
 - **Notifications:** Multi-channel support
-

Verified Real Implementations

AUTH-001: Authentication System (VERIFIED REAL)

File: [internal/auth/auth.go](#) **Lines:** 470 **Assessment:** Production-ready

Verified capabilities:

- User registration with validation
- Password hashing with bcrypt + argon2
- JWT token generation and verification
- Session management with crypto-random tokens
- Password verification with fallback chain
- Constant-time comparison for timing attack prevention

Agent Note: This is the gold standard. All other packages should match this quality.

Verified Bluff Areas (MUST FIX)

BLUFF-001: LLM Generation is Simulated (CRITICAL)

File: `cmd/cli/main.go` lines 190-214 **Evidence:**

```
// For now, simulate generation
// In production, this would use the actual LLM provider
response := fmt.Sprintf("Generated response for: %s\n\nThis is a simulated
response...", prompt)
```

Impact: Users following README's documented CLI usage receive fake responses.

Fix Priority: P0 - Immediate

BLUFF-002: Model Listing is Hardcoded (CRITICAL)

File: `cmd/cli/main.go` lines 101-128 **Evidence:** Only 3 hardcoded models. No dynamic discovery.

Fix Priority: P0 - Immediate

BLUFF-003: Command Execution is Simulated (HIGH)

File: `cmd/cli/main.go` lines 237-250 **Evidence:**

```
// For now, simulate command execution
fmt.Printf("Executing: %s\n", command)
time.Sleep(1 * time.Second)
fmt.Printf("Command completed successfully\n")
```

Fix Priority: P0 - Immediate

Technology Stack

Core Technologies:

- **Language:** Go 1.24.0 with toolchain go1.24.9
- **Module:** `dev.helix.code`
- **HTTP Framework:** Gin v1.11.0
- **Authentication:** JWT v4.5.2
- **Database:** PostgreSQL 15+ (optional)
- **Cache:** Redis 7+ (optional)
- **Configuration:** Viper v1.21.0
- **CLI Framework:** Cobra v1.8.0
- **Testing:** Testify v1.11.1

UI Technologies:

- **Desktop:** Fyne v2.7.0
- **Terminal UI:** tvview v0.42.0
- **Mobile:** gomobile bindings

External Integrations:

- **Browser Automation:** chromedp v0.14.2
- **Web Scraping:** goquery v1.10.3
- **Tree-sitter:** go-tree-sitter

Essential Build Commands

CRITICAL: All commands must be run from the `HelixCode/` subdirectory.

Core Commands

- **Build:** `make build` (generates logo assets and builds to bin/helixcode)
- **Test all:** `make test` or `go test -v ./...`
- **Test single:** `go test -v -run TestName ./path/to/package`
- **Lint:** `make lint` or `golangci-lint run ./...`
- **Format:** `make fmt` or `go fmt ./...`
- **Clean:** `make clean`

Development Workflow

- **Dev server:** `make dev`
- **Logo assets:** `make logo-assets` (required before first build)
- **Setup deps:** `make setup-deps` or `go mod tidy`

Specialized Builds

- **Production:** `make prod` (cross-platform)
- **Mobile:** `make mobile` (iOS + Android)
- **Aurora OS:** `make aurora-os`
- **Harmony OS:** `make harmony-os`

Testing Variations

- **Unit:** `./run_tests.sh --unit`
- **Integration:** `./run_tests.sh --integration`
- **E2E:** `./run_tests.sh --e2e`
- **Coverage:** `./run_tests.sh --coverage`
- **Security:** `./run_tests.sh --security`
- **Challenges:** `cd tests/e2e/challenges && ./run_all_challenges.sh`

Architecture & Code Organization

Core Structure

```

HelixCode/
├── cmd/                # Application entry points
│   ├── server/        # Main HTTP server
│   ├── cli/           # CLI client
│   └── security-test/ # Security testing tools
├── internal/          # Internal packages
│   ├── auth/          # JWT authentication (VERIFIED REAL)
│   ├── llm/           # LLM providers (BLUFF AREA)
│   ├── worker/        # SSH-based worker pool
│   ├── task/          # Task management
│   ├── server/        # HTTP server & API
│   ├── database/      # PostgreSQL layer
│   ├── tools/         # Tool ecosystem
│   ├── editor/        # Code editing
│   ├── memory/        # Memory integration
│   ├── notification/  # Notifications
│   ├── mcp/           # MCP protocol
│   ├── workflow/      # Workflow engine
│   └── ...
├── applications/      # Platform-specific apps
├── tests/             # Test suites
└── config/            # Configuration files

```

Configuration Management

Primary Configuration

Main config at `config/config.yaml`:

```

server:
  address: "0.0.0.0"
  port: 8080

database:
  host: "" # Empty to disable for testing
  port: 5432
  user: "helix"
  dbname: "helixcode_prod"
  sslmode: "disable"

redis:
  host: "redis"
  port: 6379
  enabled: true

auth:
  jwt_secret: "change-in-production"
  token_expiry: 86400
  session_expiry: 604800

```

```
workers:
  health_check_interval: 30
  max_concurrent_tasks: 10

tasks:
  max_retries: 3
  checkpoint_interval: 300

llm:
  default_provider: "local"
  max_tokens: 4096
  temperature: 0.7
```

Environment Variables

Required for Production:

- `HELIX_DATABASE_PASSWORD`
- `HELIX_AUTH_JWT_SECRET`
- `HELIX_REDIS_PASSWORD`

LLM Provider Keys:

- `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GEMINI_API_KEY`, etc.

Testing Approach

Test Categories

1. **Unit tests:** Mocks allowed, `*_test.go`, `-short` flag
2. **Contract tests:** Real API schemas, no mocks
3. **Component tests:** Real subsystems wired together
4. **Integration tests:** Full app with real dependencies
5. **E2E challenges:** Complete user workflows
6. **Security tests:** OWASP compliance
7. **Performance tests:** Benchmarks

Anti-Bluff Testing Rules

- Unit tests: Mocks OK
- ALL other tests: Real infrastructure ONLY
- Every PASS guarantees Quality + Completion + Usability
- Challenges fail on simulated/stubbed behavior
- No bare `t.Skip()` without `SKIP-OK: #<ticket>`

Key Subsystems

LLM Package (`internal/llm/`)

Status: CRITICAL BLUFF AREA

Target state:

- 15+ providers with real implementations
- Provider manager with fallback chain
- Circuit breakers per provider
- Health checks
- Model discovery

Tools Package ([internal/tools/](#))

Status: To be verified

Target tools:

- Filesystem: fs_read, fs_write, fs_edit, glob, grep
- Shell: shell, shell_background with sandbox
- Web: web_fetch, web_search
- Browser: browser_launch, browser_navigate, browser_screenshot
- Git: git automation
- MultiEdit: Transactional multi-file editing

Editor Package ([internal/editor/](#))

Status: To be verified

Target formats:

- Diff Format (unified diff)
- Whole File replacement
- Search/Replace with regex
- Line-Based edits

Free AI Providers

- **XAI (Grok):** grok-3-fast-beta, grok-3-mini-fast-beta
- **OpenRouter:** Free models from various providers
- **GitHub Copilot:** gpt-4o, claude-3.5-sonnet (with subscription)
- **Qwen:** 2,000 requests/day free tier

Docker Deployment

Quick Start

```
cd HelixCode
cp .env.example .env
# Edit .env with secure passwords
```

```
docker-compose up -d
docker-compose ps
curl http://localhost/health
```

Services

- helixcode-server (ports 8080, 2222)
- postgres (port 5432)
- redis (port 6379)
- nginx (ports 80, 443)

Important Gotchas

Critical Requirements

- **Always work from HelixCode/ subdirectory**
- **Generate logo assets before first build:** `make logo-assets`
- **Database/Redis optional:** Disable by setting `database.host: ""`
- **Environment variables override config file**

Anti-Bluff Mandate (CONST-035)

Every agent working on HelixCode MUST:

1. Verify code actually works before marking complete
2. Write tests that validate REAL behavior
3. Write challenges that exercise complete workflows
4. Never commit simulated/placeholder code to production paths
5. Include pasted terminal output as evidence of working code

Common Issues

1. **Build fails:** Run `make logo-assets` then `make build`
2. **Database errors:** Check `HELIX_DATABASE_PASSWORD`
3. **Worker SSH failures:** Verify SSH key authentication
4. **LLM timeouts:** Check provider status and config

Universal Mandatory Constraints

Hard Stops (permanent, non-negotiable)

1. **NO CI/CD pipelines**
2. **NO HTTPS for Git** (SSH only)
3. **NO manual container commands** (orchestrator-owned)

Mandatory Development Standards

1. **100% Test Coverage** (unit, integration, E2E, automation, security, benchmark)

2. **Challenge Coverage** (every component)
3. **Real Data** (actual API calls, real DB, live services)
4. **Health & Observability** (health endpoints, circuit breakers)
5. **Documentation & Quality** (update docs with code changes)
6. **Validation Before Release** (full suite + all challenges)
7. **No Mocks in Production**
8. **Comprehensive Verification** (runtime, compile, structure, dependencies, compatibility)
9. **Resource Limits** (30-40% of host resources max)
10. **Bugfix Documentation** (root cause, affected files, fix, verification link)
11. **Real Infrastructure for All Non-Unit Tests**
12. **Reproduction-Before-Fix** (Challenge first, then fix)
13. **Concurrent-Safe Collections**

Definition of Done

A change is NOT done because code compiles. "Done" requires:

- Pasted terminal output from a real run
- No self-certification words without evidence
- Demo commands that run against real artifacts
- Loud skips with **SKIP-OK: #<ticket>** markers

CONST-035 — End-User Usability Mandate

A test or Challenge that PASSES is a CLAIM that the tested behavior **works for the end user of the product**.

The HelixAgent project has repeatedly hit the failure mode where every test ran green AND every Challenge reported PASS, yet most product features did not actually work — buggy challenge wrappers masked failed assertions, scripts checked file existence without executing the file, "reachability" tests tolerated timeouts, contracts were honest in advertising but broken in dispatch. **This MUST NOT recur in HelixCode.**

Every PASS result MUST guarantee: a. **Quality** - correct behavior under real inputs, edge cases, concurrency b. **Completion** - wired end-to-end with no stub/placeholder gaps c. **Full usability** - a user following documentation succeeds

A passing test that doesn't certify all three is a **bluff** and MUST be tightened.

Bluff Taxonomy (each pattern observed and now forbidden)

- **Wrapper bluff** - assertions PASS but wrapper's exit-code logic is buggy
- **Contract bluff** - system advertises capability but rejects it in dispatch
- **Structural bluff** - file exists but doesn't contain working code
- **Comment bluff** - comment promises behavior code doesn't have
- **Skip bluff** - `t.Skip("not running yet")` without **SKIP-OK: #<ticket>** marker

The taxonomy is illustrative, not exhaustive. Every Challenge or test added going forward MUST pass an honest self-review against this taxonomy before being committed.

Contact & Resources

- **Constitution:** [CONSTITUTION.md](#)
- **CLAUDE.md:** [CLAUDE.md](#)
- **Gap Analysis:** [HELIXCODE_GAP_ANALYSIS.md](#)
- **Zero-Bluff Plan:** [HELIXCODE_ZERO_BLUFF_PLAN.md](#)
- **Testing Strategy:** [ANTI_BLUFF_TESTING_STRATEGY.md](#)

Built with zero-bluff commitment. Every feature actually works.